LAPORAN OPTIMASI DATABASE — MINGGU 5

Nama MHS	Adnan Anwar Rabbani
NIM	A11202415540
Kelas	……
Mata kuliah	Pemrograman Sisi Server (A11.64707) — Suprayogi, M.Kom.
Repository GitHub	……

1. Data Pengujian

Dataset dibuat memakai dua management command, yaitu `generate_performance_data` (10.000 Course berkode PERF) dan `generate_relationship_data` (user, enrollment, lesson, assignment, submission untuk 100 Course PERF pertama). Kedua command sudah dicoba dijalankan dua kali dan tidak menambah data ganda. Angka di bawah dihitung langsung dari database, sehingga sudah termasuk data seed Minggu 4.

Data	Jumlah	Keterangan
Course	10.005	10.000 Course PERF + 5 Course dari seed Minggu 4
Lecturer	6	5 dosen simulasi + 1 dosen seed
Student	51	50 mahasiswa simulasi + 1 mahasiswa seed
Lesson	503	5 per Course PERF (100 Course) + 3 dari seed
Enrollment	1.001	10 mahasiswa per Course PERF + 1 dari seed
Assignment	202	2 per Course PERF + 2 dari seed
Submission	1.002	5 per Assignment PERF + 2 dari seed

Cara pengukuran. Jumlah query dihitung dengan `CaptureQueriesContext`. Waktu adalah median dari 7 kali pengulangan setelah satu kali pemanasan (warm-up), dijalankan pada database dan data yang sama untuk BEFORE dan AFTER. Basis data: PostgreSQL 16.15. Pengukuran dilakukan lewat command `benchmark_week5` di lingkungan pengerjaan project, bukan di Docker Compose milik penulis, sehingga angka waktu bisa berbeda di komputer lain. Jumlah query dan jenis scan yang dipakai PostgreSQL adalah bukti yang tidak bergantung pada kecepatan komputer.

2. Hasil Pengujian

2.1 Baseline (Checkpoint 1)

Mengambil seluruh Course dengan `list(Course.objects.all())`. Walau ada 10.005 baris, Django hanya mengirim satu query. Jumlah query bergantung pada pola akses, bukan pada banyaknya data.

```
Jumlah Course: 10005
Jumlah query SQL: 1
```

2.2 Tabel perbandingan BEFORE dan AFTER

Percobaan	Query BEFORE	Query AFTER	Waktu BEFORE → AFTER	Kesimpulan
Course + Lecturer (100 Course)	101 (1 + 100)	1	54,782 ms → 2,632 ms (turun 95,2%)	`select_related('lecturer')` menggabungkan Course dan User dengan JOIN, jadi tidak ada query per Course.
Course + Lesson (50 Course)	51 (1 + 50)	2 (1 + 1)	42,319 ms → 6,074 ms (turun 85,6%)	`prefetch_related('lessons')` mengambil semua Lesson dalam satu query tambahan lalu memasangkannya di Python.
Database Index (is_active, title)	EXPLAIN: Seq Scan, cost 239,07	EXPLAIN: Index Scan, cost 8,31	0,568 ms → 0,022 ms (turun 96,1%)	PostgreSQL berhenti membaca seluruh tabel dan langsung memakai index, tetapi selisih waktunya baru kecil (di bawah 1 ms) pada 10.005 baris.

2.3 Course + Lecturer (select_related)

```
# BEFORE
for c in Course.objects.filter(code__startswith='PERF').order_by('code')[:100]:
    result.append((c.title, c.lecturer.username))

# AFTER
qs = Course.objects.filter(code__startswith='PERF').order_by('code').select_related('lecturer')[:100]
for c in qs:
    result.append((c.title, c.lecturer.username))
```

Pada versi BEFORE, Django mengambil 100 Course dengan satu query. Setiap pemanggilan `c.lecturer` memicu satu query lagi ke tabel User, sehingga totalnya 101 query. Pada versi AFTER, `select_related` memasukkan JOIN ke query utama dan hasilnya 1 query. Cara ini cocok karena `lecturer` adalah ForeignKey: tiap Course hanya punya satu dosen, jadi JOIN tidak menggandakan baris. Waktu median turun dari 54,782 ms menjadi 2,632 ms.

2.4 Course + Lesson (prefetch_related)

```
# BEFORE
for c in Course.objects.filter(code__startswith='PERF').order_by('code')[:50]:
    titles = [l.title for l in c.lessons.all()]

# AFTER
qs = Course.objects.filter(code__startswith='PERF').order_by('code').prefetch_related('lessons')[:50]
for c in qs:
    titles = [l.title for l in c.lessons.all()]
```

Satu Course punya banyak Lesson (reverse ForeignKey), jadi `c.lessons.all()` di dalam loop menjalankan satu query untuk setiap Course: 51 query untuk 50 Course. Dengan `prefetch_related` Django menjalankan dua query saja, yaitu satu untuk Course dan satu untuk seluruh Lesson dari Course tersebut (`WHERE course_id IN (...)`). JOIN kurang cocok di sini karena baris Course akan berulang sebanyak Lesson-nya. Jumlah query tetap 2 berapa pun jumlah Course. Waktu median turun dari 42,319 ms menjadi 6,074 ms.

2.5 Teknik lain: exists(), count(), values(), annotate()

```
exists: True
count: 10000
[{'code': 'PERF00000', 'title': 'Performance Course 0'}, {'code': 'PERF00001', 'title': 'Performance Course 1'},
       {'code': 'PERF00002', 'title': 'Performance Course 2'}]
PERF00000 10
PERF00001 10
PERF00002 10
```

Keempatnya masing-masing hanya memakai 1 query SQL. `exists()` dan `count()` tidak memuat objek Course ke memori, `values()` hanya mengambil kolom yang diperlukan, dan `annotate(Count('enrollments'))` menyerahkan perhitungan jumlah mahasiswa ke database (setiap Course PERF yang diuji berisi 10 mahasiswa).

2.6 Database index

Index ditambahkan pada class `Meta` model Course, lalu dimigrasikan (`0003_course_catalog_cou_is_acti_d7f8f8_idx`):

```
class Meta:
    ordering = ["code"]
    indexes = [
        models.Index(fields=["is_active", "title"]),
    ]
```

Agar perbandingan adil, kondisi BEFORE dibuat dengan melepas index sementara pada tabel dan data yang sama, lalu memasangnya kembali untuk kondisi AFTER (diikuti ANALYZE). Query yang diuji: `Course.objects.filter(is_active=True, title='Performance Course 5001').explain(analyze=True)`.

EXPLAIN ANALYZE sebelum index:

```
Sort  (cost=239.07..239.08 rows=1 width=58) (actual time=0.540..0.541 rows=1 loops=1)
  Sort Key: code
  Sort Method: quicksort  Memory: 25kB
  ->  Seq Scan on catalog_course  (cost=0.00..239.06 rows=1 width=58) (actual time=0.267..0.537 rows=1 loops=1)
        Filter: (is_active AND ((title)::text = 'Performance Course 5001'::text))
        Rows Removed by Filter: 10004
Planning Time: 0.037 ms
Execution Time: 0.549 ms
```

EXPLAIN ANALYZE sesudah index:

```
Sort  (cost=8.31..8.32 rows=1 width=58) (actual time=0.013..0.013 rows=1 loops=1)
```

```
  Sort Key: code
  Sort Method: quicksort  Memory: 25kB
  ->  Index Scan using catalog_cou_is_acti_d7f8f8_idx on catalog_course  (cost=0.29..8.30 rows=1 width=58)
      (actual time=0.009..0.010 rows=1 loops=1)
        Index Cond: ((is_active = true) AND ((title)::text = 'Performance Course 5001'::text))
Planning Time: 0.044 ms
Execution Time: 0.022 ms
```

Ukuran	Sebelum index	Sesudah index
Jenis scan	Seq Scan (Rows Removed by Filter: 10004)	Index Scan memakai catalog_cou_is_acti_d7f8f8_idx
Cost (perkiraan)	239,07	8,31
Execution Time (EXPLAIN ANALYZE)	0,568 ms	0,022 ms
Waktu query lewat ORM (median)	1,143 ms	0,503 ms

Catatan temuan. Query pada modul memakai judul *Performance Course 5000*. Generator memberi `is_active=False` pada course dengan nomor kelipatan 5, sehingga course 5000 tidak aktif dan query tersebut mengembalikan 0 baris. Rencana query-nya tetap berubah dari Seq Scan menjadi Index Scan (0,603 ms → 0,018 ms), tetapi untuk bukti yang lebih jelas pengujian utama memakai judul aktif 5001 yang mengembalikan 1 baris.

Pembanding: kolom `code` sudah `unique=True`, jadi PostgreSQL sudah memiliki index untuknya dan `filter(code='PERF05000')` langsung memakai Index Scan (0,013 ms).

```
Index Scan using catalog_course_code_e109f879_like on catalog_course  (cost=0.29..8.30 rows=1 width=58) (actual
      time=0.006..0.006 rows=1 loops=1)
  Index Cond: ((code)::text = 'PERF05000'::text)
Planning Time: 0.028 ms
Execution Time: 0.013 ms
```

2.7 Pengujian aplikasi

`docker compose exec app python manage.py test catalog` — seluruh test, termasuk 3 test wajib Minggu 4 dan test baru untuk query count, index, serta generator data, tetap lulus setelah optimasi.

```
GeneratorTests.test_generators_create_expected_data_and_are_repeatable ... ok
IndexTests.test_index_declared_on_model ... ok
IndexTests.test_index_exists_in_database ... ok
QueryOptimizationTests.test_exists_count_values_annotate ... ok
QueryOptimizationTests.test_n_plus_1_lecturer_before ... ok
QueryOptimizationTests.test_n_plus_1_lessons_before ... ok
QueryOptimizationTests.test_prefetch_related_lessons_after ... ok
QueryOptimizationTests.test_select_related_lecturer_after ... ok
CourseTestCase.test_course_code_unique ... ok
CourseTestCase.test_course_created ... ok
CourseTestCase.test_lesson_relationship ... ok
ExtraModelTests.test_custom_manager_active ... ok
ExtraModelTests.test_default_role_is_student ... ok
ExtraModelTests.test_enrollment_unique ... ok
ExtraModelTests.test_lesson_order_unique_per_course ... ok
ExtraModelTests.test_seed_demo_is_idempotent ... ok
ExtraModelTests.test_select_related_uses_one_query ... ok
ExtraModelTests.test_submission_relationship ... ok

Ran 18 tests in 7.198s

OK
```

Migrasi yang terpasang: `catalog 0001_initial, 0002_initial, 0003_course_catalog_cou_is_acti_d7f8f8_idx`.

3. Kesimpulan

Optimasi yang dilakukan. Ada tiga: `select_related()` untuk relasi Course → Lecturer, `prefetch_related()` untuk relasi Course → Lesson, dan index gabungan `(is_active, title)` pada Course. Setiap perubahan diukur sebelum dan sesudah pada data yang sama (10.005 Course).

Apakah query berkurang? Ya. Course + Lecturer turun dari 101 menjadi 1 query, dan Course + Lesson dari 51 menjadi 2 query. Pada kode awal jumlah query bertambah mengikuti jumlah Course (1 + N); setelah optimasi jumlahnya tetap, sehingga selisihnya makin besar kalau data bertambah. Waktu median ikut turun, dari 54,782 ms ke 2,632 ms dan dari 42,319 ms ke 6,074 ms.

Index. Rencana query berubah dari Seq Scan menjadi Index Scan dan cost turun dari 239,07 ke 8,31. Execution time turun dari 0,568 ms ke 0,022 ms. Dalam angka mutlak selisihnya tidak sampai 1 ms karena tabel hanya berisi sekitar sepuluh ribu baris. Manfaat nyata baru terasa kalau tabel jauh lebih besar. Index juga memakan ruang penyimpanan dan membuat INSERT, UPDATE, serta DELETE sedikit lebih berat. Jadi index sebaiknya dibuat untuk pola query yang memang sering dipakai.

Manfaat bagi LMS. Halaman yang menampilkan daftar course beserta dosen dan materinya akan tetap ringan ketika jumlah course bertambah, karena beban ke database tidak ikut naik per course. Hasil ini juga menjadi dasar yang baik untuk pembuatan REST API di Minggu 6.

Lampiran — Screenshot Django Shell

Tempelkan screenshot dari komputer sendiri pada kotak berikut (hasil menjalankan perintah di `docker compose exec app python manage.py shell`).

Checkpoint 1 — jumlah Course dan jumlah query

BEFORE — Course + Lecturer (N+1)

AFTER — Course + Lecturer (select_related)

BEFORE dan AFTER — Course + Lesson (prefetch_related)

EXPLAIN sebelum dan sesudah index, serta hasil migrate

Hasil python manage.py test catalog